$ npm install --save-dev hardhat@latest

$ npx hardhat --init

Solidity智能合约开发

第11.1课：Hardhat 3 环境搭建

专业开发框架

TypeScript支持

Ignition部署

Creating a professional development environment for Solidity smart contracts

- Node.js v22+
- TypeScript support
- Ignition deployment system

NEW 第三阶段 | 开发框架与实战 | Hardhat 3

区块链导师：【Layer】

从Remix到Hardhat：专业开发框架的必要性

Remix的局限

浏览器IDE，适合学习和简单测试，功能有限

优点：

- 零配置，开箱即用
- 适合学习和快速测试
- 可视化界面友好

局限：

- 缺乏版本控制
- 无法自动化测试
- 不支持复杂项目结构
- 团队协作困难
- 无法集成CI/CD

Hardhat的优势

专业开发框架，完整开发工作流，企业级项目必备

专业特性：

- 完整的本地开发环境
- 强大的测试框架
- 自动化部署脚本
- 丰富的插件生态
- Git版本控制
- 团队协作友好
- 主网/测试网无缝切换

学习路径： Remix（入门学习） → Hardhat（专业开发） → 企业级项目

Hardhat 2 vs Hardhat 3

Why Choose Hardhat 3?

特性	Hardhat 2	Hardhat 3 NEW
Node.js版本要求	>= 14	>= 22
默认Solidity版本	0.8.19	0.8.24
TypeScript支持	需手动配置	开箱即用
部署系统	脚本/hardhat-deploy	Ignition
插件安装	单独安装	Toolbox 统一安装
网络管理	基础支持	多链支持 EDR引擎

选择Hardhat 3的理由：

- **现代化架构：**EDR运行时引擎；更好性能和稳定性
- **Ignition部署系统：**声明式定义；自动状态管理
- **更好的开发体验：**TypeScript开箱即用；强大类型支持
- **官方推荐：**Hardhat团队主推版本；社区支持活跃

Hardhat 3 核心功能

Core Features

内置测试网络

Hardhat Network (EDR)
本地区块链模拟
零配置启动

智能合约测试

Mocha + Chai
Solidity测试支持
Gas消耗分析

NEW

Ignition部署系统

声明式部署定义
自动状态管理
跨链部署支持

传统脚本部署

脚本化部署
多网络支持
灵活控制

合约验证

Etherscan自动验证
源码透明化
多网络支持

调试工具

console.log调试
堆栈跟踪
Gas分析

插件生态

Gas Reporter, Etherscan验证, Typechain, OpenZeppelin升级

本节课学习概要

Learning Path

① **Node.js环境准备**
安装Node.js v22+和npm

② **Hardhat 3项目初始化**
创建并初始化Hardhat项目

③ **项目结构详解**
理解contracts、scripts、test等目录

④ **配置文件与网络配置**
掌握hardhat.config.ts和本地/测试网络配置

⑤ **第一个合约部署**
完成合约编译和部署

Node.js和npm

JavaScript Runtime & Package Manager

Node.js

- JavaScript运行时环境
- 基于Chrome V8引擎
- 异步非阻塞I/O
- 跨平台
- 后端服务

npm

- Node Package Manager
- 包管理工具
- 依赖安装和管理
- 版本控制
- 脚本运行

Hardhat 3 版本要求

Node.js >= 22.0.0（必须！） npm会随Node.js自动安装 推荐使用LTS版本

Node.js安装 – Windows

1、下载安装包

- 访问Node.js官方网站
- 选择LTS版本（推荐）或Current版本

> ⚠ 重要提示：确保版本 ⩾ 22.0.0

2、运行安装程序

- 双击下载的.msi安装文件
- 接受许可协议
- 选择安装路径（默认即可）
- **勾选"Add to PATH"**
- 完成安装

3、验证安装

- 打开命令提示符（cmd）或PowerShell

```
$ node --version
输出应为 v22.x.x 或更高
$ npm --version
输出应为 10.x.x 或更高
```

提示： 确保Node.js安装在系统PATH中，以便在任何位置使用命令行工具。

Node.js安装 – macOS/Linux

macOS安装方式

方式1：官网下载

访问 nodejs.org
下载.pkg安装包
双击安装

方式2：Homebrew（推荐）

```
# 安装Homebrew（如未安装）
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
# 安装Node.js（最新LTS）
brew install node@22
# 验证安装
node --version
npm --version
```

必须 >= v22.0.0

Linux安装方式

Ubuntu/Debian

使用NodeSource仓库（推荐）

```
# 使用NodeSource仓库
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -

# 安装Node.js
sudo apt-get install -y nodejs
# 验证安装
node --version
npm --version
```

必须 >= v22.0.0

创建Hardhat 3项目

Creating Your First Hardhat 3 Project

1

创建项目目录

```
$ mkdir my-hardhat3-project
$ cd my-hardhat3-project
```

2

初始化npm

```
$ npm init -y
# 创建 package.json 文件
```

3

安装Hardhat 3

```
$ npm install --save-dev hardhat@latest

# 下载Hardhat 3及其依赖
```

4

初始化项目

```
$ npx hardhat --init
```

5

安装依赖

```
$ npm install
```

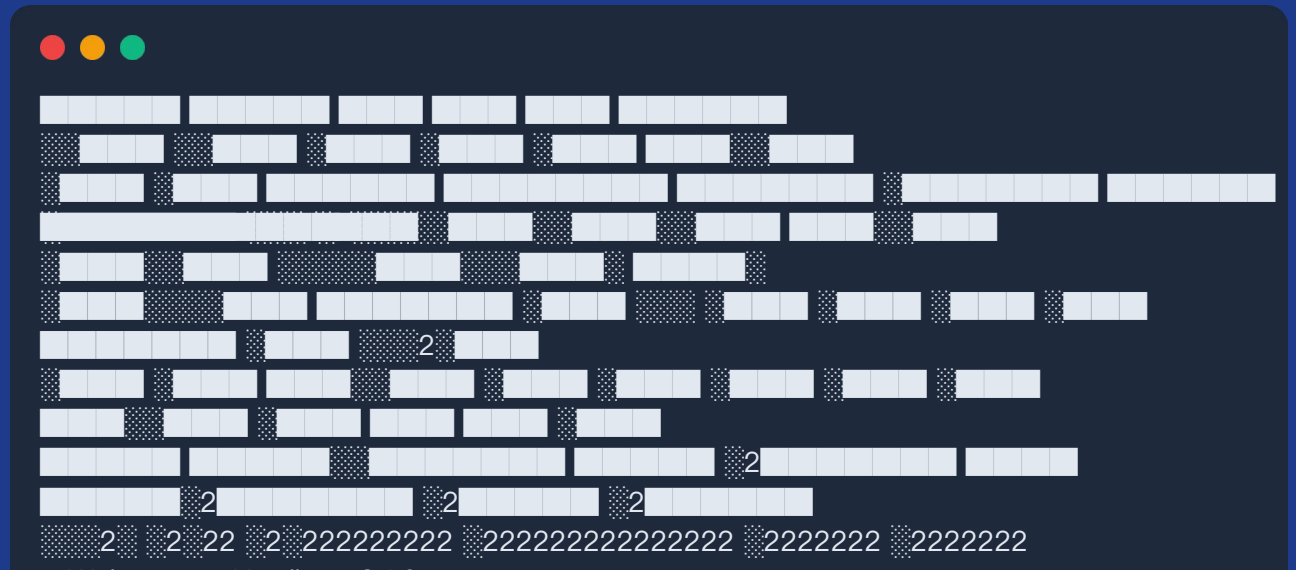
推荐选择TypeScript配置：

在初始化项目时，**选择"Create a TypeScript project"**以获得更好的开发体验和类型支持。

安装过程完成后，项目将包含所有必要的文件和目录结构。

Hardhat 3 初始化过程详解

Understanding the Initialization Process



创建的文件说明（Hardhat 3）

- my-hardhat3-project/
- contracts/
  - 示例合约：Counter.sol
  - 测试文件：Counter.t.sol
- scripts/
  - 示例脚本：send-op-tx.ts
- test/
  - 测试文件：Counter.ts
- **ignition/**
  - 模块目录：modules/
  - 示例模块：Counter.ts
- hardhat.config.ts
  - Hardhat配置文件
- package.json
  - npm配置
- tsconfig.json
  - TypeScript配置

Hardhat 3 项目结构详解 Project Structure Deep Dive

目录树

- contracts/
  - Counter.sol
  - Counter.t.sol
- scripts/
  - send-op-tx.ts
- test/
  - Counter.ts
- ignition/
  - modules/
  - Counter.ts

contracts/

存放所有Solidity智能合约
- .sol文件和.t.sol测试文件
- 示例：Counter.sol

scripts/

部署和交互脚本目录
- 使用ethers.js/viem编写
- 适合简单场景

test/

单元测试文件目录
- TypeScript/JavaScript或Solidity测试
- 使用Mocha + Chai

ignition/

Hardhat Ignition部署模块
- 声明式部署定义
- 自动状态管理

contracts目录详解

```sol
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

contract Counter {
  uint public x;

  event Increment(uint by);

  function inc() public {
    x++;
    emit Increment(1);
  }

  function incBy(uint by) public {
    require(by > 0, "incBy: increment should be positive");
    x += by;
    emit Increment(by);
  }
}
```

合约功能说明

- 计数器合约（Counter）：存储和递增数字
- 核心功能：存储一个数字；设置数字；递增数字
- 学习要点：状态变量；函数定义；public可见性

Hardhat 3 特点

- 默认Solidity 0.8.24
- 支持.t.sol测试文件
- 更好的编译优化
- 智能合约目录自动识别

🚀 scripts目录详解

Understanding Scripts Directory

send-op-tx.ts

```
import hre from "hardhat";

async function main() {
  // 获取签名者
  const [signer] = await hre.ethers.getSigners();
  console.log("Deploying contracts with account:",
signer.address);

  // 获取余额
  const balance = await hre.ethers.provider.getBalance
(signer.address);
  console.log("Account balance:", hre.ethers.formatEther
(balance));

  // 部署合约 (Hardhat 3 API)
  const counter = await hre.ethers.deployContract("Counter");
  await counter.waitForDeployment();

  const address = await counter.getAddress();
  console.log("Counter deployed to:", address);
}

main()
  .then(() => process.exit(0))
```

部署流程（Hardhat 3）

- 导入Hardhat运行环境：`import hre from "hardhat"`
- 获取签名者：`getSigners()`
- 获取余额：`ethers.provider.getBalance()`
- 部署合约：`deployContract()` NEW
- 等待部署确认：`waitForDeployment()`
- 获取合约地址：`getAddress()` NEW

test目录详解 Understanding Test Directory

测试目录结构

- test/ – 单元测试文件目录
- TypeScript/JavaScript或Solidity测试
- 使用Mocha + Chai测试框架

运行测试

- 命令: `npx hardhat test`
- 可选: 运行特定测试文件
- Gas报告: `REPORT_GAS=true npx hardhat test`

测试文件示例（TypeScript）

```typescript
import { expect } from "chai";
import { network } from "hardhat";

const { ethers } = await network.connect();

describe("Counter", function () {
  it("Should emit the Increment event when calling the inc() function", async function () {
    const counter = await ethers.deployContract("Counter");

    await expect(counter.inc()).to.emit(counter, "Increment").withArgs(1n);
  });

  it("The sum of the Increment events should match the current value", async function () {
    const counter = await ethers.deployContract("Counter");
    const deploymentBlockNumber = await ethers.provider.getBlockNumber();

    // run a series of increments
    for (let i = 1; i <= 10; i++) {
      await counter.incBy(i);
    }

    const events = await counter.queryFilter(
      counter.filters.Increment(),
      deploymentBlockNumber,
      "latest",
    );

    // check that the aggregated events match the current value
    let total = 0n;
    for (const event of events) {
      total += event.args.by;
    }

    expect(await counter.x()).to.equal(total);
  });
});
```

ignition目录详解 Understanding Ignition Directory

Ignition模块示例

```
import { buildModule } from
"@nomicfoundation/hardhat-
ignition/modules";

export default
buildModule("CounterModule", (m)
=> {
  const counter =
m.contract("Counter");

  m.call(counter, "incBy", [5n]);

  return { counter };
});
```

核心特点

- 声明式部署定义 – 通过模块化方式定义部署逻辑
- 自动状态管理 – 跟踪已部署合约，避免重复部署
- 错误恢复支持 – 部署失败后可从中断处继续
- 依赖自动处理 – 自动识别并按正确顺序部署合约依赖
- 跨链部署支持 – 支持在不同链上进行部署

NEW

目录结构

- ignition/
  - modules/
    - Counter.ts
  - parameters/
    - sepolia.json
    - mainnet.json
  - deployments/
    - chain-11155111/

hardhat.config.ts配置

Configuration File Deep Dive (Hardhat 3)

```
import { HardhatUserConfig } from "hardhat/config";
import "@nomicfoundation/hardhat-toolbox";
const config: HardhatUserConfig = {
// Solidity编译器版本
solidity: "0.8.24",
// 网络配置（Hardhat 3新格式）
networks: {
hardhat: {
      type: "edr-simulated",
      chainId: 31337,
      },
      },
};
export default config;
```

- 必须指定type字段
- "edr-simulated" – 本地网络

zSolidity编译器配置指南

单版本配置

```
// simple config
constconfig:HardhatUserConfig= {
 {
   solidity: "0.8.24"
};
```

- 简洁明了的版本指定，适用于单一版本项目

多版本配置

```
// multi-version config
constconfig:HardhatUserConfig = {
   solidity: {
      compilers: [
         {
            version: "0.8.19",
            settings: {
               optimizer: {
                  enabled: true,
                  runs: 200
               }
            }
         },
         {
            version: "0.7.6"
         }
      ]
   }
};
```

- 支持多个Solidity版本同时编译
- 可为每个版本单独配置优化器

编译优化器

enabled:
- true – 启用优化（推荐）
- false – 禁用优化

runs:
- 200 – 适中（默认）
- 1 – 优化部署成本

使用场景

开发： 可关闭优化，编译更快

生产： 必须启用优化，降低Gas

网络配置详解（Hardhat 3）

配置代码

标准配置

```ts
// Hardhat 3 标准配置
import { HardhatUserConfig } from "hardhat/config";
import "@nomicfoundation/hardhat-toolbox";
import "dotenv/config";

const config: HardhatUserConfig = {
  solidity: {
    version: "0.8.24",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200,
      },
    },
  },
  networks: {
    hardhat: {
      type: "edr-simulated",  // 🆕 必须
      chainId: 31337,
    },
    sepolia: {
      type: "http",  // 🆕 必须
      url: process.env.SEPOLIA_URL || "",
      accounts: process.env.PRIVATE_KEY
        ? [process.env.PRIVATE_KEY] : [],
      chainId: 11155111,
    },
    mainnet: {
      type: "http",
      url: process.env.MAINNET_URL || "",
      accounts: process.env.PRIVATE_KEY
        ? [process.env.PRIVATE_KEY] : [],
      chainId: 1,
    },
  },
  gasReporter: {
    enabled: process.env.REPORT_GAS !== undefined,
    currency: "USD",
  },
  etherscan: {
    apiKey: process.env.ETHERSCAN_API_KEY,
  },
};
```

高级配置

```ts
// Hardhat 3 高级配置
import hardhatToolboxMochaEthersPlugin
  from "@nomicfoundation/hardhat-toolbox-mocha-ethers";
import { configVariable, defineConfig }
  from "hardhat/config";

export default defineConfig({
  plugins: [hardhatToolboxMochaEthersPlugin],
  solidity: {
    profiles: {
      default: {
        version: "0.8.28"
      },
      production: {
        version: "0.8.28",
        settings: {
          optimizer: {
            enabled: true,
            runs: 200,
          },
        },
      },
    },
  },
  networks: {
    hardhatMainnet: {
      type: "edr-simulated",
      chainType: "l1",  // 🆕
    },
    hardhatOp: {
      type: "edr-simulated",
      chainType: "op",  // 🆕
    },
    sepolia: {
      type: "http",
      chainType: "l1",
      url: configVariable("SEPOLIA_RPC_URL"),
      accounts: [configVariable("SEPOLIA_PRIVATE_KEY")],
    },
  },
});
```

配置详解

基础

导入模块

- HardhatUserConfig - TypeScript 类型
- @nomicfoundation/hardhat-toolbox - 核心工具包
- dotenv/config - 自动加载 .env

Solidity 配置

- version: "0.8.24" - Hardhat 3 默认
- optimizer.enabled: true - 启用优化
- optimizer.runs: 200 - 平衡优化（推荐）

Gas 报告

- enabled - 环境变量控制
- currency: "USD" - 货币单位
- 使用: REPORT_GAS=true npx hardhat test

Etherscan 验证

- apiKey - Etherscan API 密钥
- 命令: npx hardhat verify ——network sepolia ADDRESS

网络

网络配置（Hardhat 3 新特性）

🆕 必须指定 type 字段

1. 本地网络（hardhat）

- type: "edr-simulated" - EDR 引擎
- chainId: 31337 - 默认链 ID

2. 测试网络（sepolia）

- type: "http" - HTTP RPC
- chainId: 11155111 - Sepolia

3. 主网（mainnet）

- type: "http" - HTTP RPC
- chainId: 1 - 以太坊主网

高级特性

- chainType: "l1" - 以太坊主网/L1
- chainType: "op" - Optimism L2
- configVariable() - 智能配置变量
- profiles - 多编译配置

⚠ 错误

常见错误与修正

错误	正确
type: undefined	type: "edr-simulated"
hardhat: { chainId: 31337 }	hardhat: { type: "edr-simulated", chainId: 31337 }
sepolia: { url: "..." }	sepolia: { type: "http", url: "..." }
networks: { hardhat: {} }	networks: { hardhat: { type: "edr-simulated" } }

环境变量配置

Environment Variables Setup

创建.env文件

```
# .env文件示例
# RPC节点
SEPOLIA_URL=https://sepolia.infura.io/v3/your-project-
MAINNET_URL=https://mainnet.infura.io/v3/your-project-id
# 私钥（  切勿泄露！）
PRIVATE_KEY=your-private-key-here# API Keys
ETHERSCAN_API_KEY=your-etherscan-api-key
```

安装dotenv

```
npm install dotenv --save-dev
```

在config中使用（Hardhat 3）

```
import { HardhatUserConfig } from "hardhat/config";
import "@nomicfoundation/hardhat-toolbox";
import "dotenv/config";
# 自动加载.env
const config: HardhatUserConfig = {
  networks: {
    sepolia: {
      type: "http",
      url: process.env.SEPOLIA_URL || "",
      accounts: process.env.PRIVATE_KEY ?
      [process.env.PRIVATE_KEY] : [],
    }
  }
};
```

安全最佳实践

- 创建.env文件存储敏感信息
- 添加.env到.gitignore

常用Hardhat 3命令

Essential Hardhat 3 Commands

</>编译相关

- `npx hardhat compile` 编译合约
- `npx hardhat clean && npx hardhat compile` 清理缓存重新编译

测试相关

- `npx hardhat test` 运行所有测试
- `npx hardhat test test/Counter.ts` 运行特定测试文件
- `REPORT_GAS=true npx hardhat test` 显示Gas报告

部署相关 NEW

- `npx hardhat run scripts/send-op-tx.ts` 部署到本地网络
- `npx hardhat run scripts/send-op-tx.ts --network sepolia` 部署到Sepolia测试网
- `npx hardhat ignition deploy ignition/modules/Counter.ts` 部署Ignition模块

Ignition部署演示

Hardhat Ignition Deployment Demo

1 创建Ignition模块

```
// ignition/modules/Counter.ts
import { buildModule }
from"@nomicfoundation/hardhat-
ignition/modules";
export default buildModule("CounterModule", (m)
=> {
    const counter = m.contract("Counter");

    m.call(counter, "incBy", [5n]);
    return { counter };
});
```

2 部署到本地网络

```
$ npx hardhat ignition deploy
ignition/modules/Counter.ts
Ignition  Deploying [ CounterModule ]
Contract deployed to:
0x5FbDB2315678afecb367f032d93F642f64180aa
3
```

3 查看部署状态

```
$ npx hardhat ignition status chain-31337

Deployment: CounterModule - Status:
Completed
```

4 部署到测试网

```
$ npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia

    部署成功!
```

Ignition优势

- 自动状态管理
- 错误恢复支持
- 依赖自动处理

实战演示

Hands-on Demo

1 </> 编译合约

```
$ npx hardhat compile
Compiling 1 file with 0.8.24
Compilation finished successfully
• artifacts/ – 编译产物
```

2 运行测试

```
$ npx hardhat test
Counter
    Should emit the Increment event
2 passing (1s)
```

3 部署合约与验证

```
$ npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost
Counter deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
$ npx hardhat ignition status chain-31337
Deployment: CounterModule – Status:      Completed
```

课程总结

Summary & Next Steps

本节回顾

- Hardhat 3框架介绍与优势
- 开发环境准备（Node.js v22+, npm）
- 项目初始化与结构理解（contracts/, scripts/, test/）
- 配置文件详解（hardhat.config.ts, 网络配置）

关键收获

- 从Remix到Hardhat 3的转型实现专业开发工作流
- 掌握Ignition部署系统，现代化的声明式部署方式
- 完整的开发环境：编译、测试、部署一体化
- 多网络支持：轻松切换开发/测试/生产环境